

4. Класове и обекти в C++.

Членове на клас (член-данни и член-функции). Конструктори и деструктори. Константни член-данни. Статични членове на клас. Капсулация. Видимост и време на живот на обектите. Примерен код на C++.

1. Обектно-ориентирано програмиране

ООП-то е парадигма, при която програмата е моделира като съвкупност от обекти, които взаимодействат помежду си. C++ е хибриден език - поддържа едновременно процедурен и ООП стил.

Трите закона на ООП:

- **Капсулация** - данните и функциите, работещи с тях, са обединени в един клас
- **Наследяване** - нов клас може да разшири или специализира съществуващ
- **Полиморфизъм** - обекти от различни класове могат да се обработват по еднакъв начин чрез общ интерфейс

2. Клас и обект

Класът е дефиниция (шаблон, тип), която описва структурата и поведението на обектите. **Обектът** е конкретен екземпляр (instance) на даден клас - той заема реална памет по време на изпълнение.

2.1 Синтаксис за дефиниция на клас

```
class ИмеНаКлас {  
    // членове  
}; // задължително ;
```

2.2 Деклариране и използване на обект

```
BankAccount acc1("Иван", 1000.0); // обект в стека  
BankAccount* acc2 = new BankAccount("Мария", 500.0); //обект  
в динамичната памет  
delete acc2;
```

3. Членове на клас

Класът може да съдържа два вида членове:

- Член-данни (data members) - Променливи, описващи състоянието на обекта
- Член-функции (member function / методи) - Функции, описващи поведението на обекта

3.1 Член-данни

Декларира се точно като обикновенни променливи, но вътре в тялото на класа. Всеки обект получава собствено копие на член-данните (с изключение на статичните).

```
class Point {  
    int x;
```

```
int y;  
};
```

3.2 Член-функция

Описват операциите над обекта. Могат да бъдат:

- Дефинирани inline - в тялото на класа
- Дефинирани извън класа - с оператора за обхват ::

```
class Point {  
public:  
    int x;  
    int y;  
  
    double instance() const {  
        return sqrt(x * x + y * y);  
    }  
  
    void print() const;  
};  
  
void Point::print() const {  
    cout << "( " << x << ", " << y << " )" << endl;  
}
```

Указателят `this` - всяка нестатична член-функция получава имплицитен параметър `this`, указател към обекта, върху който е извикана. Използва се при конфликт на имена или при връщане на самия обект (`return *this`).

4. Спецификатори на достъп - капсулация

C++ предоставя 3 нива на видимост:

- **public** - Достъп отвсякъде
- **protected** - Достъп от самия клас + наследници
- **private** - Достъп само в класа

По подразбиране членовете на class са `private`, а на struct са `public`.

Капсулацията означава, че член-данните се скриват като `private`, а достъпът до тях се осигурява само чрез публични методи (getters и setters). Това предпазва обекта от неправилно използване:

```
class Temperature {
private:
    double celsius; // скрито

public:
    void set(double c) {
        if (c < -273.15) return;
        celsius = c;
    }

    double get() const { return celsius; }
    double toFahrenheit() const { return celsius * 9.0/5.0 +
32; }
};
```

5. Конструктори и деструктори

5.1 Конструктор

Конструкторът е специална член-функция, която:

- Носи същото име като класа
- Няма тип на връщана стойност (дори `void`)
- Извиква се автоматично при създаване на обект

Видове конструктори:

Конструктор по подразбиране

```
class Box {  
public:  
    double w, h, d;  
    Box() : w(1.0), h(1.0), d(1.0) {}  
}
```

Конструктор с параметри

```
Box(double width, double height, double depth)  
    : w(width), h(height), d(depth) {}
```

Конструктор за копиране

Създава нов обект като копие на съществуващ.

Сигнатура: `ClassName(const ClassName& other)` .

```
Box(const Box& other)  
    : w(other.w), h(other.h), d(other.d) {}
```

Ако класът управлява динамична памет, **задължително** се пише собствен copy constructor (за да се избегне плитко копиране).

Списък за инициализация (Member Initializer List)

Предпочита се пред присвояване в тялото, защото инициализира директно (по-ефективно, задължително за const и референтни членове):

```
Box(double w, double h, double d) : w(w), h(h), d(d) {}
```

5.2 Деструктор

Деструкторът:

- Носи същото име като класа, предшествано от `~`
- Няма параметри и тип на връщана стойност
- Извиква се автоматично при унищожаване на обекта
- Основна употреба: освобождаване на динамично заделена памет

```
class DynArray {
private:
    int* data;
    int size;

public:
    DynArray(int n) : size(n) {
        data = new int[n];
    }

    ~DynArray() {
        delete[] data;    // задължително!
    }
};
```

Правило на три (Rule of Three): Ако дефинирате деструктор, copy constructor или оператор `=` - дефинирайте и трите.

6. Константни член-данни

Декларират се с ключова дума `const`. Стойността им се задава само в списъка за инициализация на конструктора и не може да бъде променена след това.

```
class Circle {
private:
    const double PI = 3.14159265;    // константа на ниво клас
    const int id;                    // различна за всеки обект
};
```

```

double        radius;

public:
    Circle(int i, double r) : id(i), radius(r) {}

    double area() const {
        return PI * radius * radius;
    }
};

```

`const` член-функции — функция, маркирана с `const` след параметрите, не може да модифицира нестатичните член-данни. Трябва да се използва за всички read-only методи:

```

double getRadius() const { return radius; } // правилно
void    setRadius(double r) const { radius = r; } // ГРЕШКА –
не се компилира

```

7. Статични членове на клас

7.1 Статични член-данни (`static`)

Споделени от всички обекти на класа - съществува само едно копие, независимо колко обекта са създадени. Трябва да се дефинират и инициализират **извън** класа:

```

class Counter {
private:
    static int count; // декларация
public:
    Counter() { count++; }
    ~Counter() { count--; }
    static int getCount() { return count; }
};

```

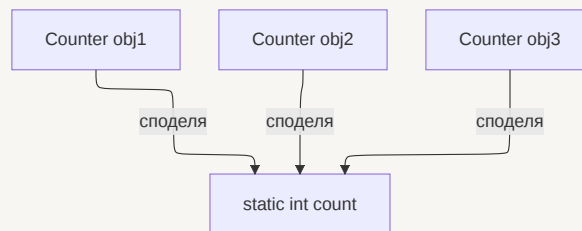
```
int Counter::count = 0; // дефиниция и инициализация извън
класа
```

graph TD

A[Counter obj1] -->|споделя| S["static int count"]

B[Counter obj2] -->|споделя| S

C[Counter obj3] -->|споделя| S



7.2 Статични член-функции

- Могат да се извикат без обект: `Counter::getCount()`
- Няма достъп до `this` и до нестатични членове
- Използват се за достъп до статични данни или фабрични методи

```
int main() {
    Counter a, b, c;
    cout << Counter::getCount(); // извежда 3
}
```

Характеристика	Нестатичен член	Статичен член
Копие за всеки обект	✓	✗ (едно за класа)
Достъп чрез обект	✓	✓
Достъп без обект	✗	✓ (<code>Клас::член</code>)
Достъп до <code>this</code>	✓	✗

8. Видимост и време на живот на обектите

8.1 Обекти в стека

```
void foo() {  
    Point p(3.0, 4.0);    // създаден при влизане  
    p.print();  
}    // <-- деструкторът се извиква автоматично тук
```

- Живеят в областта, в която са декларирани (блок, функция)
- Унищожават се автоматично при излизане от областта
- Бързо заделяне, но ограничен размер

8.2 Обекти в динамичната памет

```
Point* p = new Point(3.0, 4.0); // явно създаване  
// ...  
delete p; // явно унищожаване  
p = nullptr;
```

- Живеят докато програмистът не извика `delete`
- Гъвкавост (неизвестен брой обекти), но риск от изтичане на памет (memory leak) при забравен `delete`

8.3 Глобални и статични обекти

```
static BankAccount reserve("НБУ", 0);    // статичен  
// живее до края на програмата
```

- Конструкторът се извиква преди `main()` (за глобални)
- Деструкторът се извиква след `main()`

9. Пример - пълен клас **BankAccount**

```
#include <iostream>
#include <string>
using namespace std;

class BankAccount {
private:
    string owner;
    double balance;
    const int id;           // константна член-данна
    static int nextId;      // статична член-данна

public:
    // Конструктор с параметри + списък за инициализация
    BankAccount(const string& name, double initialBalance)
        : owner(name), balance(initialBalance), id(nextId++)
    {
        cout << "Сметка #" << id << " създадена за " << owner
        << endl;
    }

    // Конструктор за копиране
    BankAccount(const BankAccount& other)
        : owner(other.owner), balance(other.balance), id(next
    Id++) {
        cout << "Копие на сметка #" << other.id << " -> #" <<
    id << endl;
    }

    // Деструктор
    ~BankAccount() {
        cout << "Сметка #" << id << " затворена." << endl;
    }
}
```

```

// Методи за достъп (const – не модифицират обекта)
string getOwner() const { return owner; }
double getBalance()const { return balance; }
int getId() const { return id; }

// Методи за модификация
void deposit(double amount) {
    if (amount > 0) balance += amount;
}

bool withdraw(double amount) {
    if (amount > 0 && balance >= amount) {
        balance -= amount;
        return true;
    }
    return false;
}

// Статичен метод – извиква се без обект
static int getTotalAccounts() { return nextId; }

void print() const {
    cout << "Сметка #" << id << " | " << owner
        << " | Баланс: " << balance << " лв." << endl;
}

};

// Дефиниция на статичната член-данна
int BankAccount::nextId = 1;

int main() {
    cout << "=== Обекти в стека ===" << endl;
    {
        BankAccount acc1("Иван Петров", 1000.0);
        BankAccount acc2("Мария Иванова", 500.0);
    }
}

```

```

    acc1.deposit(200.0);
    acc2.withdraw(100.0);

    acc1.print();
    acc2.print();

    cout << "Общо сметки: " << BankAccount::getTotalAccounts() << endl;

    BankAccount acc3(acc1);    // copy constructor
    // или BankAccount acc3 = acc1;
    acc3.print();
}    // <- деструкторите за acc1, acc2, acc3 се извикват тук

cout << "\n=== Обект в динамичната памет ===" << endl;
BankAccount* acc4 = new BankAccount("Георги Стоянов", 2000.0);
acc4->deposit(500.0);
acc4->print();
delete acc4;    // <- деструкторът се извиква тук

return 0;
}

```